

ME315_Lab03_Elisa

Lab03

Conjuntos de dados estruturados costumam ser divididos em múltiplas tabelas. A razão para isso é a minimização de duplicação de informação. Por este motivo, uma tarefa habitualmente realizada em manipulação de bases de dados é a combinação de dados de diferentes origens.

Carregando pacotes

```
library(readr)
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

`filter`, `lag`

Os seguintes objetos são mascarados por 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.

- Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;

- Para o arquivo de vôos:

1. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;

2. Leia apenas 1 milhão de linhas por vez;

3. Remova vôos em que o aeroporto de destino comece com a letra 1;

4. Remova registros em que existam pelo menos uma coluna faltante;

5. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;

6. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

Resolução

```
library(readr)
library(dplyr)
```

```
airlines <- read_csv("airlines.csv")
```

Rows: 14 Columns: 2

— Column specification —————

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

❗ Use `spec()` to retrieve the full column specification for this data.

❗ Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
airports <- read_csv(
  "airports.csv",
  col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
)
```

Rows: 322 Columns: 5

— Column specification —————

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

❗ Use `spec()` to retrieve the full column specification for this data.

❗ Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
stats_acumuladas <- tibble(
  DESTINATION_AIRPORT = character(),
  soma_atraso = numeric(),
  n = numeric()
)

processa_chunk <- function(chunk, pos) {

  chunk_limpo <- chunk %>%
    # 3. Remover voos cujo aeroporto de destino comece com "1"
    filter(!grepl("^1", DESTINATION_AIRPORT)) %>%
    # 4. Remover registros com colunas faltantes
    na.omit()

  stats_chunk <- chunk_limpo %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_atraso = sum(ARRIVAL_DELAY),
      n = n(),
      .groups = "drop"
    )

  stats_acumuladas <-<- bind_rows(stats_acumuladas, stats_chunk) %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_atraso = sum(soma_atraso),
```

```

      n = sum(n),
      .groups = "drop"
    )
  }

read_csv_chunked(
  "flights.csv",
  callback = SideEffectChunkCallback$new(processa_chunk),
  chunk_size = 1e6,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

```

NULL

```

# 6. Estatísticas suficientes combinadas da média de atraso por aeroporto
flights <- stats_acumuladas %>%
  mutate(atraso_medio = soma_atraso / n)

flights

```

```

# A tibble: 322 × 4
  DESTINATION_AIRPORT soma_atraso      n atraso_medio
  <chr>                <dbl> <dbl>         <dbl>
1 ABE                  12874  2220          5.80
2 ABI                   9077  2224          4.08
3 ABQ                106415 18953          5.61
4 ABR                  -2227   657         -3.39
5 ABY                   7501   864          8.68
6 ACK                   2393   487          4.91
7 ACT                  16392  1541         10.6
8 ACV                  12259  1267          9.68
9 ACY                  41744  3530         11.8
10 ADK                  -284    89         -3.19
# i 312 more rows

```

2) Selecione a operação apropriada *join* para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

1. Identifique a coluna que é a chave na tabela `flights`;
2. Identifique a coluna que é a chave na tabela `airports`;
3. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?
4. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
5. Armazene a tabela resultante no objeto `flights`.

Resolução

```
# 2.1 chave em flights
# DESTINATION_AIRPORT

# 2.2 chave em airports
# IATA_CODE

aeroportos_ausentes <- anti_join(
  flights, airports,
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
aeroportos_ausentes
```

```
# A tibble: 0 × 4
#   4 variables: DESTINATION_AIRPORT <chr>, soma_atraso <dbl>, n <dbl>,
#   atraso_medio <dbl>
```

```
# 2.4 / 2.5 - join explícito e armazenamento em `flights`
flights <- flights %>%
  left_join(
    airports %>% select(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )
```

3) Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
aeroportos_por_estado <- airports %>%
  group_by(STATE) %>%
  summarise(n_aeroportos = n()) %>%
  arrange(desc(n_aeroportos))

aeroportos_por_estado
```

```
# A tibble: 54 × 2
  STATE n_aeroportos
  <chr>    <int>
1 TX         24
2 CA         22
3 AK         19
4 FL         17
5 MI         15
6 NY         14
7 CO         10
8 MN          8
9 MT          8
10 NC          8
#   44 more rows
```

4) Apresente um mapa representando todos os atrasos observados por aeroporto.

1. Carregue o pacote `leaflet`;

2. Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
3. Armazene o grafico em b) numa variável chamada `this`;

Resolução

```
# 4.1
library(leaflet)

# 4.2 / 4.3
this <- leaflet(data = flights) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste0(
      "Aeroporto: ", DESTINATION_AIRPORT, "<br>",
      "Cidade: ", CITY, ", ", STATE, "<br>",
      "Atraso médio: ", round(atraso_medio, 1), " min"
    )
  )
```

Warning in validateCoords(lng, lat, funcName): Data contains 3 rows with either missing or invalid lat/lon values and will be ignored

this



